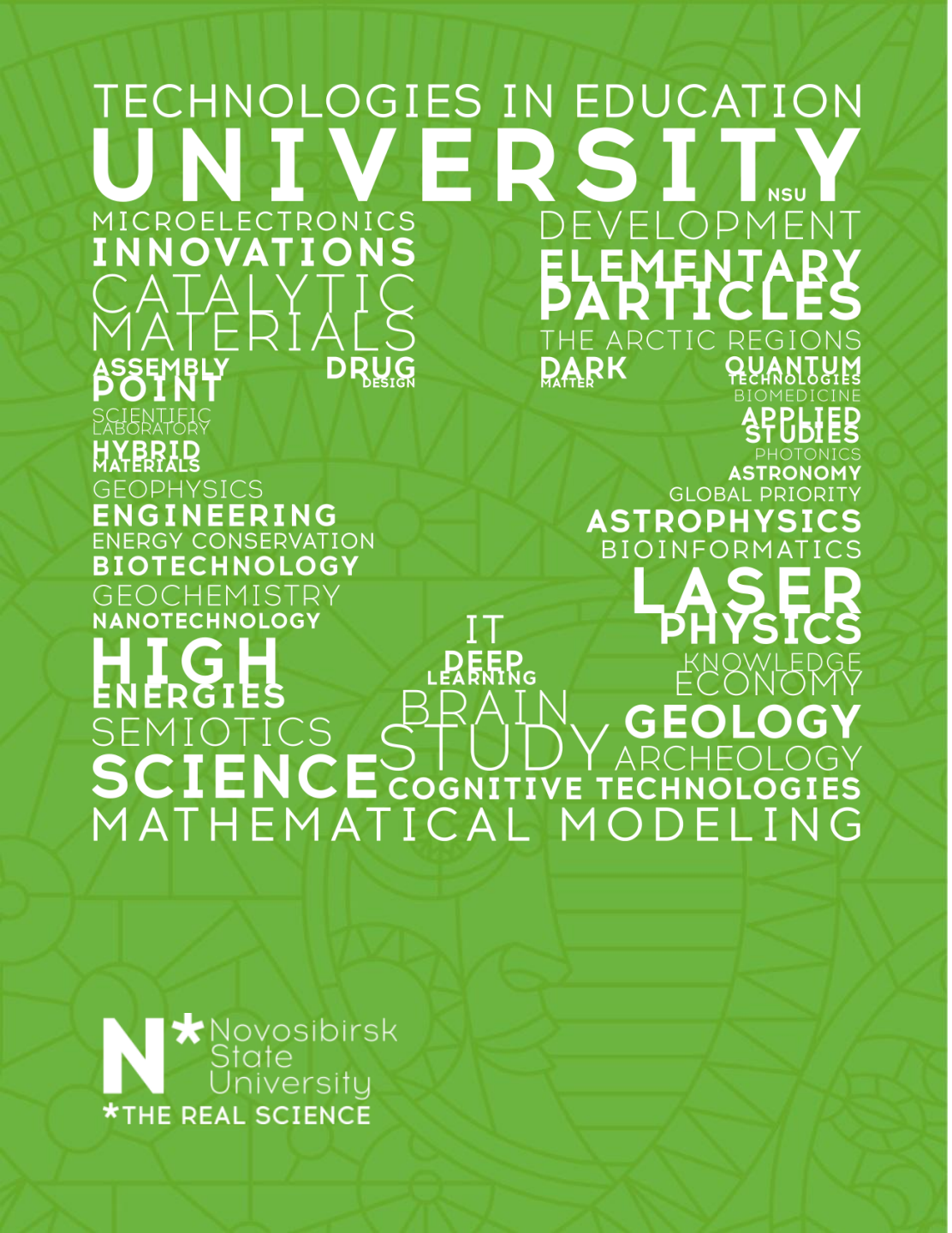


Машинное обучение
Семинар 8

Глушенко Андрей Валерьевич
Институт Интеллектуальной Робототехники НГУ

Практические аспекты

Информационная энтропия

Логические методы классификации

Деревья решений

ID3

C4.5

Информационная энтропия (Шеннон)

Клод Шеннон в 1948 году предложил формулу для измерения количества информации в сообщении. Если событие происходит со стопроцентной вероятностью, оно не несет новой информации (энтропия = 0). Чем неожиданнее событие, тем больше информации мы получаем.

$$H = -\sum_{i=1}^{k} p_i \log_2 p_i \,,$$

p_i — вероятность исхода i

Пример: Если монета честная (50/50), энтропия максимальна (1 бит). Мы понятия не имеем, что выпадет. Если монета с двумя орлами, энтропия равна 0 — результат известен заранее.

Связь с физической энтропией

Понятия очень похожи, и это не случайно.

В термодинамике (Больцман): Энтропия S характеризует количество микросостояний, которыми может быть реализовано данное макросостояние. Больше хаоса — выше энтропия.

Связь: Людвиг Больцман вывел формулу $S = k \ln \Omega$. Шеннон использовал ту же математическую структуру.

Суть: Физическая энтропия — это «недостаток информации» о точных координатах частиц в системе. Информационная энтропия — это «недостаток информации» о точном значении случайной величины.

Логические методы классификации

Логические методы классификации — это семейство алгоритмов машинного обучения, которые принимают решения на основе набора логических правил (предикатов) вида «ЕСЛИ <условие>, ТО <класс>».

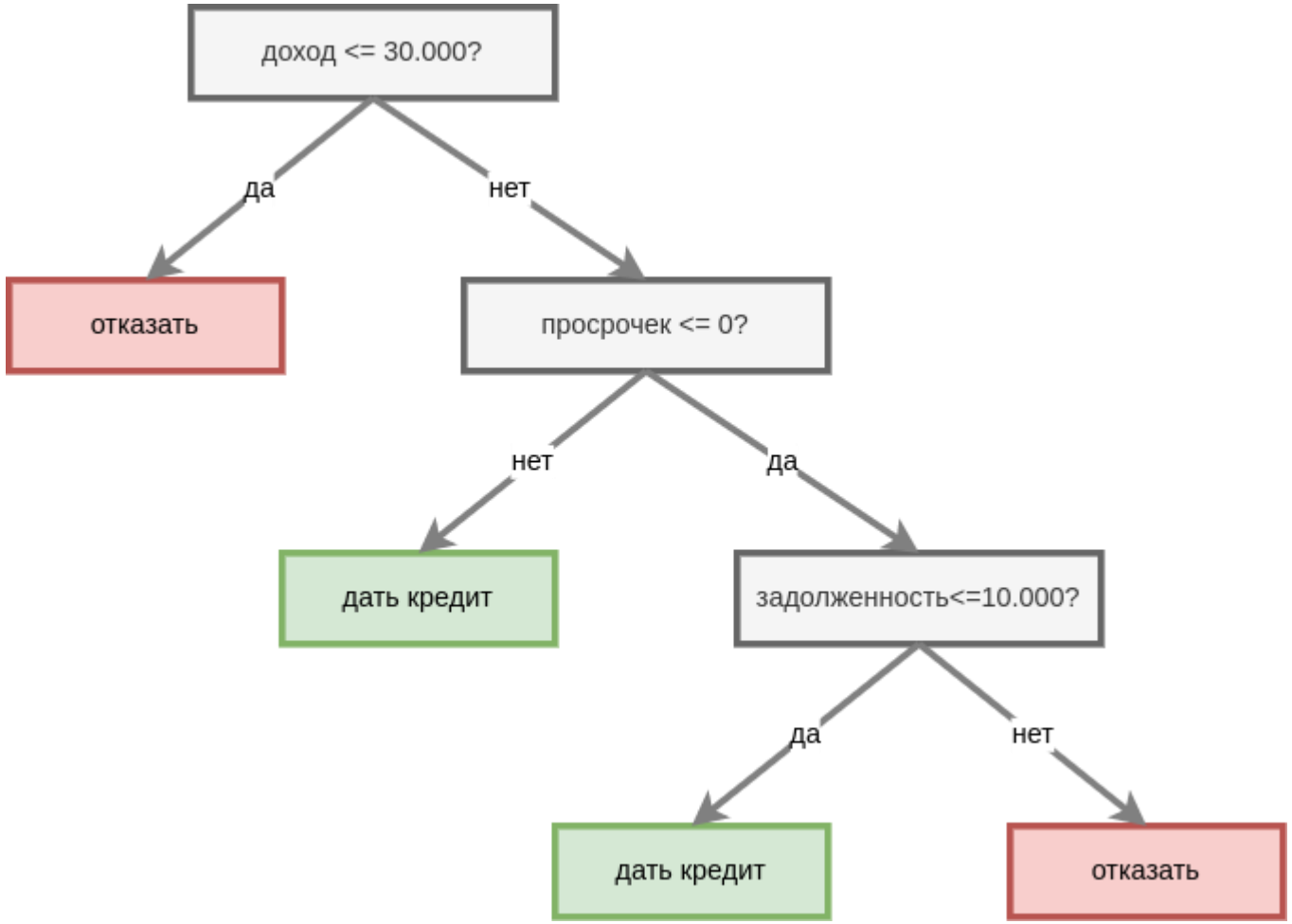
Главная особенность этих методов — интерпретируемость: человек может легко понять, почему модель приняла то или иное решение.

Логические методы классификации

ЕСЛИ <черно-белые полосы>, ТО <класс ЗЕБРА>

ЕСЛИ <сплошной коричневый окрас>, ТО <класс ДОМАШНЯЯ ЛОШАДЬ>

Деревья решений



Критерии качества решающих деревьев

Для построения эффективного решающего дерева необходимо на каждом этапе выбирать такое разбиение данных, которое максимально увеличивает «чистоту» получаемых групп. Для этого используются **критерии информативности** (меры неопределенности), которые оценивают качество распределения классов в узле.

Энтропийный критерий (Entropy)

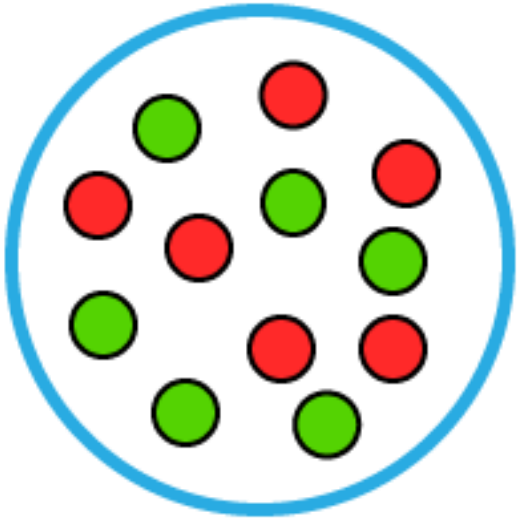
Основан на понятии информационной энтропии Шеннона и измеряет степень хаоса или неопределенности в данных.

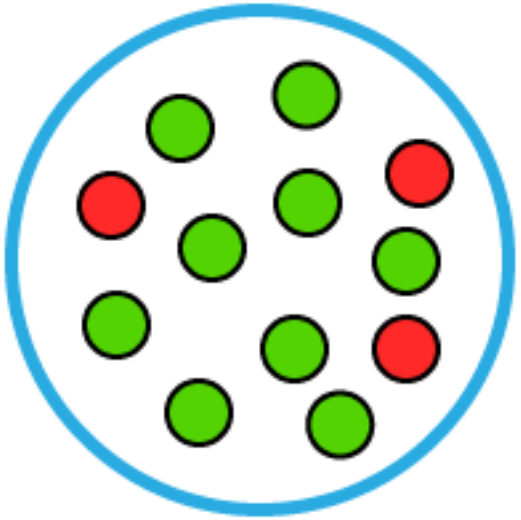
$$H = -\sum_{i=1}^{k} p_i \log_2 p_i \,,\ p_i \ - \text{доля объектов класса } i$$

- 0 — это чистый узел. Значение 1 соответствует максимальному беспорядку (для двух классов) (состояние узла, в котором объекты разных классов перемешаны в **равных пропорциях**.).
- Более чувствителен к изменениям в распределении классов, чем критерий Джини, но требует больше вычислительных ресурсов. Используется в алгоритмах ID3 и C4.5.

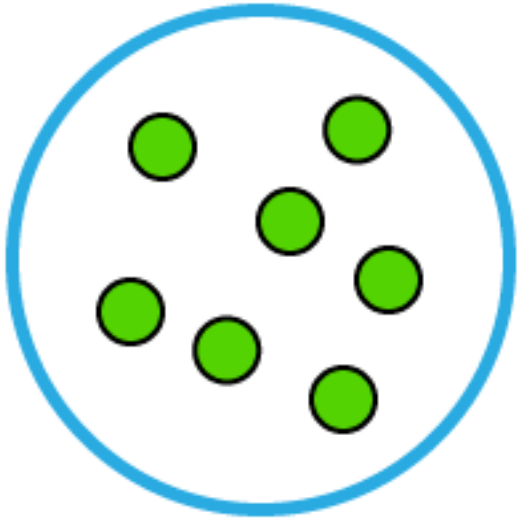
Беспорядок

Максимальный беспорядок





Чистый узел



Критерий Джини (Gini Impurity)

Индекс Джини измеряет вероятность того, что случайно выбранный элемент из узла будет классифицирован неправильно, если назначить ему метку случайным образом в соответствии с распределением классов в этом узле.

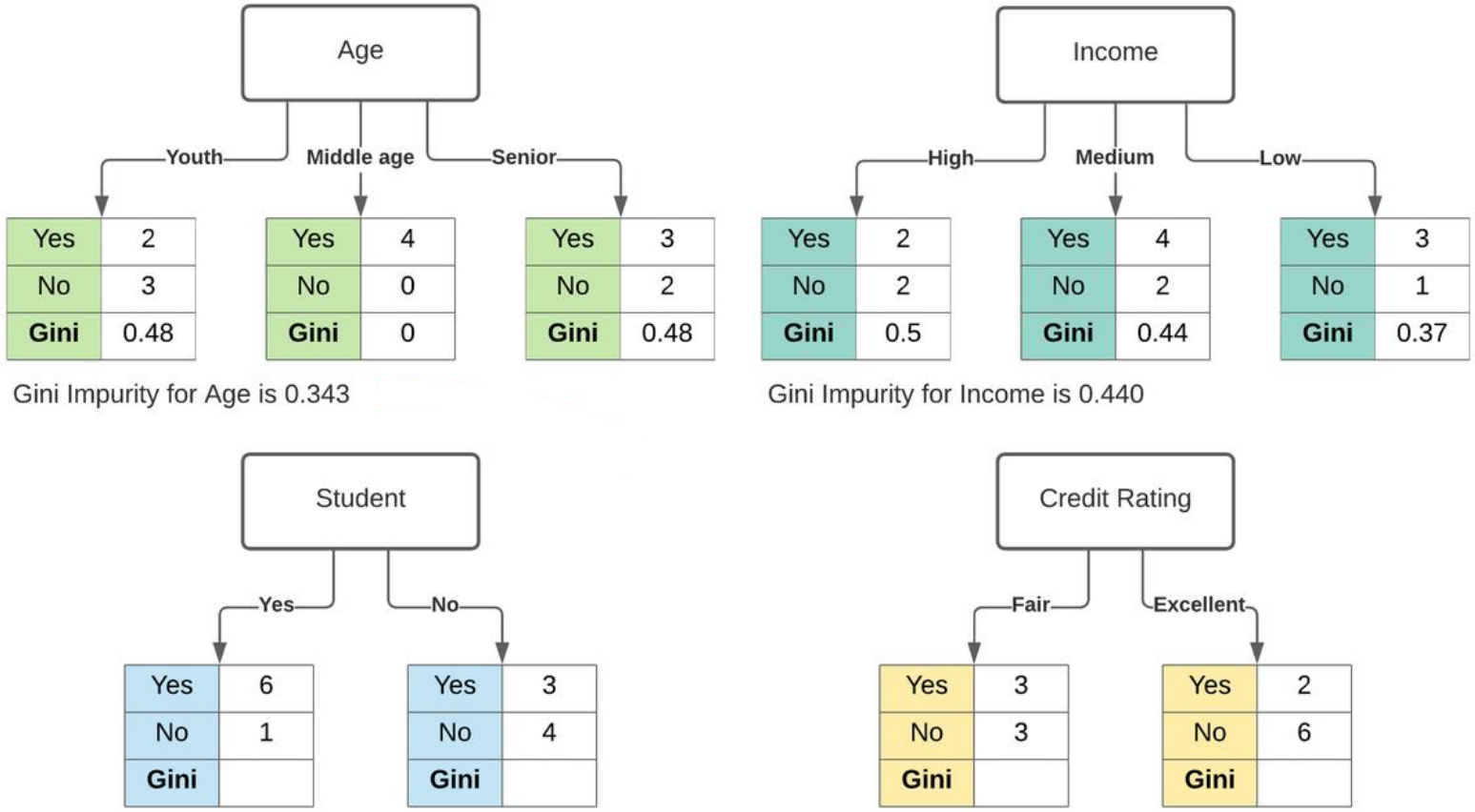
$$G = 1 - \sum_{i=1}^{k} {p_i}^2,\ p_i\ - \text{доля объектов класса } i$$

- Значение 0 означает идеальную чистоту (все объекты одного класса). Максимальное значение (0.5 для двух классов) означает равномерное распределение.
- Вычисляется быстрее энтропии, так как не требует логарифмирования. Используется в алгоритме CART.

Критерий Джини (Gini Impurity)

$$G = 1 - \sum_{i=1}^{k} {p_i}^2,\ p_i\ - \text{доля объектов класса } i$$

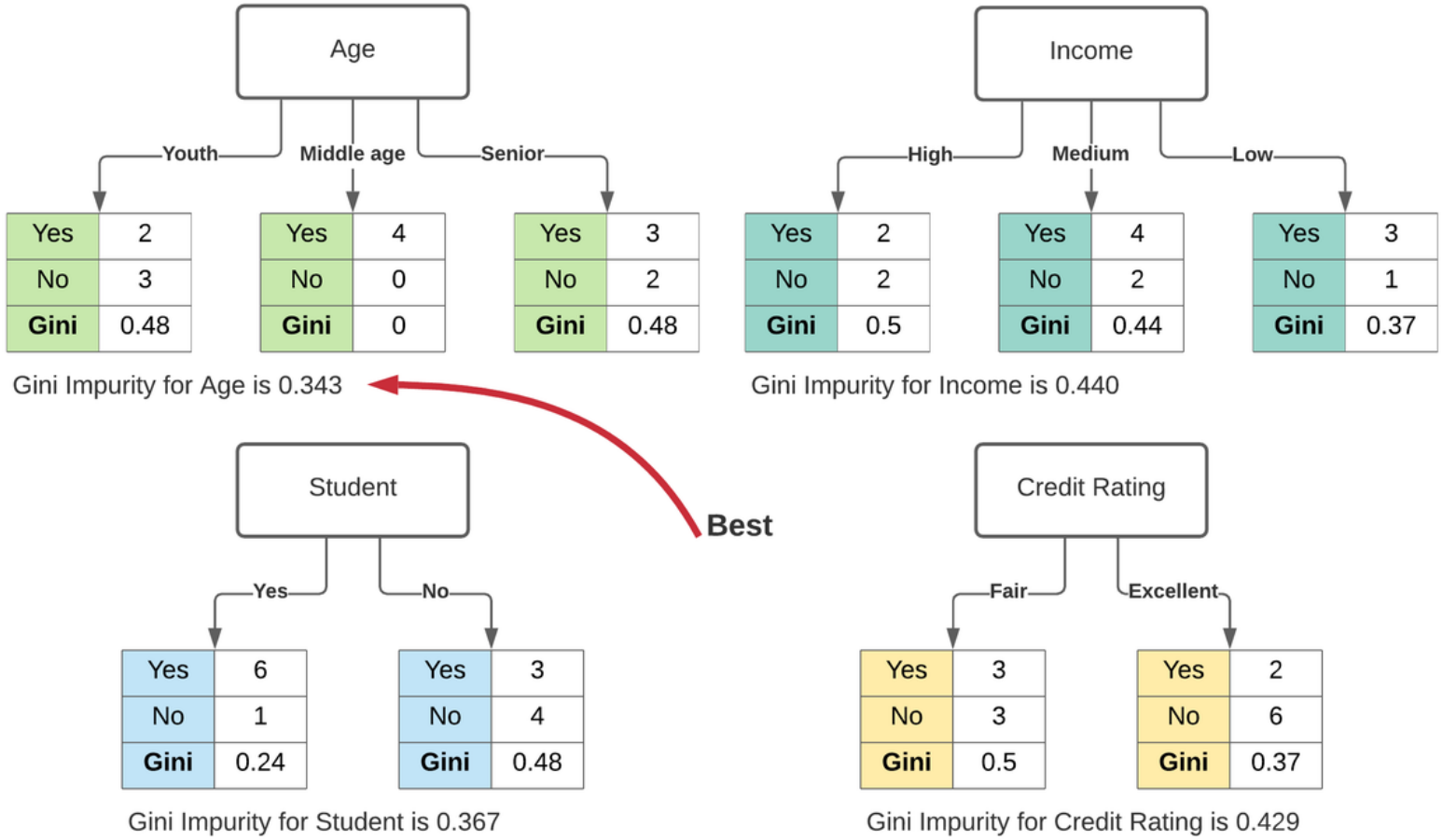
$$G_{split} = \sum_{j=1}^{k} \frac{n_j}{n} \cdot G_j$$



Критерий Джини (Gini Impurity)

$$G = 1 - \sum_{i=1}^{k} {p_i}^2,\ p_i\ - \text{доля объектов класса } i$$

$$G_{split} = \sum_{j=1}^{k} \frac{n_j}{n} \cdot G_j$$



Построение решающих деревьев жадным алгоритмом ID3

Алгоритм **ID3** (Iterative Dichotomiser 3) — это классический «жадный» алгоритм построения решающих деревьев.

Алгоритм называется жадным, потому что на каждом шаге он делает **локально оптимальный выбор**. Он ищет такой признак, который прямо сейчас лучше всего разделяет данные на классы, и не заглядывает вперед (не проверяет, к чему этот выбор приведет через 3-4 шага).

Алгоритм ID3

1. Начало: Мы берем все объекты в корневой узел.
2. Выбор лучшего признака: Для каждого доступного признака вычисляется Information Gain (Прирост информации). Алгоритм выбирает признак с максимальным значением Gain.
3. Разбиение: Узел разделяется на ветви по значениям выбранного признака.
4. Рекурсия: Процесс повторяется для каждой ветви (каждого подмножества данных).
5. Остановка: Процесс прекращается, если:
   - Все объекты в узле принадлежат одному классу (беспорядок = 0).
   - Закончились признаки для разбиения.
   - В узле больше нет объектов.

Как ID3 считает лучший признак?

Главный инструмент ID3 — Энтропия (H) и Information Gain (G).

Считаем энтропию всей группы (H_{total}): Насколько сейчас в данных всё «перемешано».

Считаем среднюю энтропию после разбиения (H_{after}): Каким станет беспорядок, если мы разделим данные по признаку «Цвет» или «Размер».

Вычисляем Прирост информации:

$$IG = H_{total} - H_{after}$$

Мы смотрим, на сколько «упал» беспорядок после использования этого признака. Чем больше упал — тем лучше признак.

Ограничения ID3 (почему появились C4.5 и CART)

- Только категориальные признаки: В оригинальном ID3 нельзя использовать числа (например, «Вес 75.5 кг»), только категории («Высокий», «Низкий»).
- Склонность к переобучению: ID3 строит очень глубокие деревья, которые идеально знают обучающую выборку, но ошибаются на новых данных.
- Проблема многозначных признаков: Если у вас есть признак «ID клиента» (где каждое значение уникально), ID3 выберет его как идеальный, так как в каждой ветке будет по 1 чистому объекту. Но это бесполезно для предсказаний.

Чтобы решающее дерево не превратилось в «шпаргалку», которая просто заучила тренировочные данные (переобученная модель), применяют **усечение (pruning)**.

Для работы с неполными данными в продвинутых версиях ID3 (например, C4.5) используют специальные стратегии **обработки пропусков**.

	School ID	Name	Address	City	Subject	Marks	Rank	Grade
0	101.0	Alice	123 Main St	Los Angeles	Math	85.0	2	B
1	102.0	Bob	456 Oak Ave	New York	English	92.0	1	A
2	103.0	Charlie	789 Pine Ln	Houston	Science	78.0	4	C
3	NaN	David	101 Elm St	Los Angeles	Math	89.0	3	B
4	105.0	Eva	NaN	Miami	History	NaN	8	D
5	106.0	Frank	222 Maple Rd	NaN	Math	95.0	1	A
6	107.0	Grace	444 Cedar Blvd	Houston	Science	80.0	5	C
7	108.0	Henry	555 Birch Dr	New York	English	88.0	3	B

Усечение дерева (pruning)

Цель усечения — упростить дерево, удаляя узлы, которые не дают значимого улучшения в предсказании на новых данных.

Предварительное усечение (Pre-pruning):

Остановка роста дерева «на лету». Мы задаем правила, когда пора прекратить деление:

- Максимальная глубина: ограничить количество уровней (например, не глубже 5).
- Минимальное число объектов в узле: если в группе осталось 5-10 примеров, дальше не делим.
- Порог прироста информации: если новый признак улучшает «чистоту» меньше чем на 1%, деление не проводим.
- Минус: можно случайно пропустить важное сложное правило, которое раскрывается только через несколько шагов.

Усечение дерева (pruning)

Пост-усечение (Post-pruning):

Сначала строим дерево «на максимум» до чистых листьев, а затем идем снизу вверх и отсекаем лишние ветки.

- **Метод уменьшения ошибки (Reduced Error Pruning):** берем ветку, временно превращаем её в лист с самым частым классом и проверяем точность на отдельной (валидационной) выборке. Если точность не упала или выросла — оставляем лист.
- **Стоимостная сложность (Cost-Complexity Pruning):** алгоритм ищет баланс между ошибкой дерева и его размером (количеством листьев). Используется в алгоритме **CART**.

Обработка пропущенных значений

Если у объекта в обучающей выборке не указан какой-то признак (например, «Возраст»), классические алгоритмы (ID3) ломаются. Продвинутые версии решают это так:

На этапе обучения (построения):

- Если мы считаем прирост информации по признаку «Зарплата», а у 10% людей она не указана:
- Считаем Gain только по тем 90% объектов, где данные есть.
- Умножаем полученный результат на коэффициент 0.9 (штрафуем признак за пропуски).
- При распределении объектов по веткам, объекты с пропусками «размножаются»: они отправляются во все ветки сразу, но с определенным весом (пропорционально количеству известных объектов в каждой ветке).

Обработка пропущенных значений

Если у объекта в обучающей выборке не указан какой-то признак (например, «Возраст»), классические алгоритмы (ID3) ломаются. Продвинутые версии решают это так:

На этапе классификации (прогноза):
Если пришел новый объект без признака, по которому нужно сделать выбор в узле:

- **Идти по всем путям:** объект идет во все дочерние ветки, собирает вероятности классов из листьев, и в конце вычисляется средневзвешенный результат.
- **Суррогатные разбиения (в CART):** алгоритм заранее ищет «запасной» признак, который максимально похож на основной (например, если нет данных о доходе, можно использовать уровень образования).

Алгоритм C4.5

Работа с непрерывными (числовыми) признаками:

- ID3 умел работать только с категориями (например, «Холодно/Тепло»). C4.5 научился обрабатывать числа (например, «Температура: 22.5°C»):
- Алгоритм сортирует значения по возрастанию.
- Проверяет все возможные точки разбиения (пороги) между соседними значениями.
- Выбирает порог, который дает лучший прирост информации. Таким образом, признак превращается в бинарное условие: Температура <= 20 или Температура > 20.

Алгоритм C4.5

Переход от Information Gain к Gain Ratio:

- ID3 часто выбирал бесполезные признаки с кучей уникальных значений (например, «ID клиента» или «Номер телефона»), потому что они давали формально идеальную чистоту.
- C4.5 использует Gain Ratio (Отношение прироста): он штрафует признаки, которые дробят данные на слишком большое количество мелких групп. Это делает дерево более логичным и устойчивым.

Встроенная обработка пропусков:

- Как мы обсуждали ранее, C4.5 не игнорирует объекты с пустыми полями. Он распределяет их по ветвям пропорционально весам уже классифицированных объектов, что позволяет использовать всю доступную информацию.

Алгоритм C4.5

Пост-усечение (Pessimistic Pruning):

- C4.5 строит полное дерево, а затем «подрезает» его, заменяя лишние ветки листьями.
- Он использует пессимистическую оценку: предполагает, что на реальных данных ошибка будет выше, чем на обучающих. Если ветка не дает существенного выигрыша в точности с учетом этого «штрафа», она удаляется.

Формирование правил (Rulesets):

- C4.5 умеет превращать дерево в набор понятных логических правил (IF-THEN). После преобразования он дополнительно оптимизирует эти правила, удаляя лишние условия, что делает модель еще более человекочитаемой.

CART (Classification and Regression Trees)

CART (Classification and Regression Trees) — это классический алгоритм построения бинарных деревьев решений. Его главная фишка в том, что он универсален: подходит и для классификации (когда предсказываем категорию), и для регрессии (когда предсказываем число).

CART (Classification and Regression Trees)

1. Начало (Корень): Мы берем весь обучающий набор данных. Это наш «корневой узел».

2. Поиск лучшего разделения:

- Алгоритм перебирает все доступные признаки (например, возраст, доход, рост).
- Для каждого признака он пробует все возможные пороговые значения (например, «Доход > 50к», «Доход > 60к»).
- Цель: найти такую пару «признак + порог», которая лучше всего разделит данные на две группы («левую» и «правую»).

CART (Classification and Regression Trees)

3. Оценка качества (Критерий):

- В **регрессии**: минимизируется среднеквадратичная ошибка (**MSE**). Алгоритм ищет разделение, при котором значения в полученных ветвях будут максимально близки к своему среднему.
- В **классификации**: используется индекс **Джини** (Gini Impurity). Он показывает, насколько «смешаны» классы в узле. Чем меньше индекс, тем более «чистым» (однородным по классам) стал узел.

4. Бинарное разбиение: После выбора лучшего признака данные делятся строго на две части. В CART всегда только две ветки из одного узла.

CART (Classification and Regression Trees)

5. Рекурсия: Эти же шаги повторяются для каждого из двух новых узлов. Процесс идет «вглубь», пока не сработает условие остановки.

6. Остановка: Дерево перестает расти, если:

- Достигнута максимальная глубина.
- В узле осталось слишком мало объектов.
- Дальнейшее разбиение не дает значимого улучшения качества.

7. Финальный прогноз (Листья):

- В регрессии: среднее значение целевой переменной всех объектов в этом листе.
- В классификации: самый часто встречающийся класс (мажоритарный).

Случайные деревья и случайный лес

Случайный лес (Random Forest) — это мощный алгоритм машинного обучения, который объединяет предсказания множества отдельных «решающих деревьев», чтобы получить один более точный и стабильный результат.

Случайные деревья

Случайное дерево решений - основа метода

Представьте дерево как последовательность вопросов, на которые можно ответить «да» или «нет». Например, чтобы решить, пойдет ли человек на прогулку, дерево может спросить: «Идет ли дождь?», а затем «Какая температура?».

Проблема одного дерева: Оно склонно к «переобучению» — слишком сильно подстраивается под обучающие данные и начинает ошибаться на новых примерах.

Случайный лес

Алгоритм решает проблему переобучения, создавая целый «лес» из таких деревьев. Чтобы деревья были разными и независимыми, используются два приема случайности:

Бэггинг (Bagging): Каждое дерево обучается не на всем наборе данных, а на его случайной подвыборке. Это называется «бутстрэп» (bootstrap).

Случайные признаки: При поиске лучшего вопроса для разделения данных (сплита), алгоритм выбирает не из всех возможных характеристик, а только из случайного их подмножества.

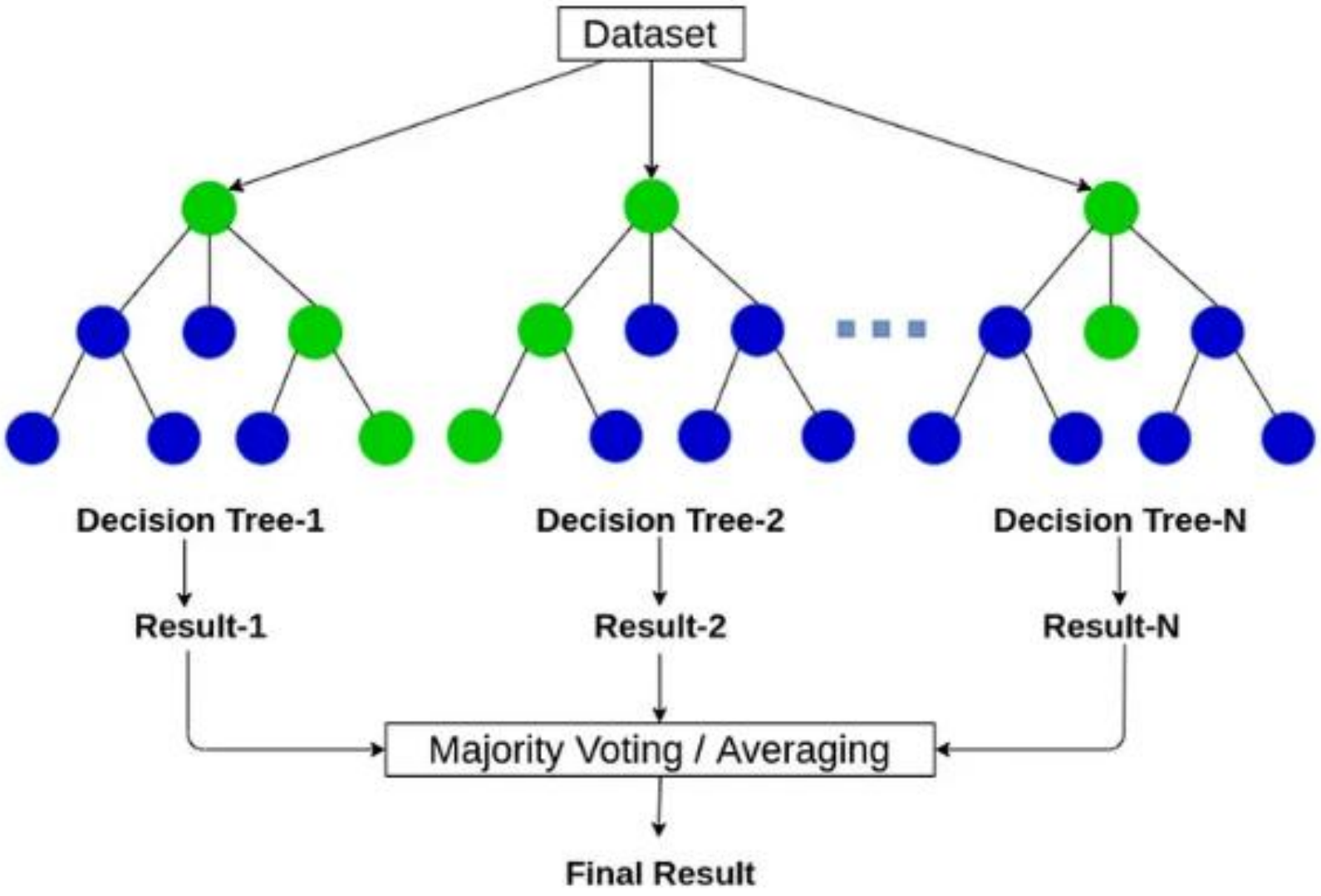
Как принимается решение?

Когда нужно сделать прогноз, каждое дерево в лесу дает свой «голос»:

В задачах классификации (например, «спам» или «не спам») побеждает вариант, за который проголосовало большинство деревьев.

В задачах регрессии (предсказание чисел, например, цены квартиры) результаты всех деревьев просто усредняются.

Случайные деревья и случайный лес



Преимущества и недостатки

Плюсы: Высокая точность, устойчивость к выбросам в данных и способность работать с большим количеством признаков без предварительной подготовки.

Минусы: Модель может занимать много памяти и работать медленнее, чем простые алгоритмы, из-за большого количества деревьев.